COLLEGE EXPERIMENT REPORT

ML Model Deployment Strategies and Model Serving using FastAPI

Submitted by
S.HemanthRegistration
Reg No.: 231FA04E40
Section: 15

This report documents the implemented Titanic model training, inference strategies, and FastAPI serving experiment.

Abstract and Objectives

This experiment demonstrates a complete introductory machine-learning deployment workflow using the official Kaggle Titanic training dataset. A scikit-learn classification pipeline is trained to estimate the target variable *Survived*. The fitted preprocessing and classifier are then reused for three inference styles: online inference for one record, batch inference for 20 records, and sequential streaming inference for 10 records. Finally, the same persisted artifact is exposed through a FastAPI REST service with Pydantic validation.

Objectives

- Prepare and validate the official Titanic training data without fabricating records or results.
- Train a reproducible scikit-learn pipeline and evaluate it on a held-out test split.
- Persist the complete preprocessing-plus-classifier pipeline with Joblib.
- Compare online, batch, and streaming inference behavior using constructed demonstration records.
- Serve predictions through validated FastAPI endpoints and exercise them with a Python client.

Official dataset source

Kaggle Titanic competition data: https://www.kaggle.com/competitions/titanic/data

Evidence note: The report uses the actual project files and recorded command outputs. Constructed inference records are demonstrations and are not claimed to be real passengers.

Dataset and Preprocessing

The source file is the official Titanic competition train.csv located in the assignment folder. It contains 891 rows and 12 columns in the verified training run. The target column is **Survived**, where 0 means did not survive and 1 means survived.

Input feature contract

Feature	Type in pipeline	Treatment
Pclass	Numeric	Median imputation
Sex	Categorical	Most-frequent imputation; one-hot encoding
Age	Numeric	Median imputation
SibSp	Numeric	Median imputation
Parch	Numeric	Median imputation
Fare	Numeric	Median imputation
Embarked	Categorical	Most-frequent imputation; one-hot encoding

Preprocessing design

A ColumnTransformer applies separate transformations to numeric and categorical columns. Numeric features are passed through a SimpleImputer using the median. Sex and Embarked are first imputed with the most frequent value and then encoded using OneHotEncoder(handle_unknown='ignore'). This allows the fitted pipeline to accept missing Age or Embarked values in later inference demonstrations.

Split and feature exclusions

The labelled data is split with train_test_split(test_size=0.2, random_state=42, stratify=y), producing an 80% training portion and a 20% held-out testing portion. PassengerId is an identifier rather than a predictive measurement. Name, Ticket, and Cabin were excluded because they were not part of the selected reproducible feature contract. Survived is the target and therefore cannot be an input feature.

Model Training and Evaluation

The implementation in train_model.py builds one scikit-learn Pipeline. Its first stage is a ColumnTransformer containing the numeric and categorical preprocessing branches described on the previous page. Its second stage is LogisticRegression(max_iter=1000). The pipeline is fitted only on X_train and y_train.

Verified training run

Measure	Observed value
Dataset rows	891
Training records	712
Testing records	179
Test accuracy	80.45%
Target distribution	0: 549; 1: 342

Accuracy is calculated with accuracy_score(y_test, predictions), where predictions are produced by applying the fitted pipeline to the held-out test features. The reported 80.45% is therefore a test-split evaluation result, not an accuracy claim about the constructed online, batch, or streaming demonstrations.

Model persistence

After fitting and evaluation, joblib.dump(model, model_path) saves the complete preprocessing-plus-classifier pipeline as titanic-ml-deployment/model.pkl. Every later inference script and the FastAPI application loads this same artifact. No later task retrains or replaces it.

Representative implementation excerpt

```
model = Pipeline([('preprocessor', preprocessor),
('classifier', LogisticRegression(max_iter=1000))])
```

Online Inference

online_inference.py resolves the project directory from __file__, loads model.pkl with Joblib, and constructs one new passenger record as a one-row pandas DataFrame. It passes that DataFrame to the loaded pipeline's predict() method. When predict_proba() is available, the script also displays estimated probabilities for both classes.

Verified constructed demonstration record

Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
2	female	29	0	1	26	C

Actual execution result

Output	Verified value
Predicted class	1
Predicted label	Survived
Estimated survival probability	84.82%
Estimated non-survival probability	15.18%

Interpretation: This is a model prediction for a newly constructed demonstration record. It is not a confirmed historical passenger outcome and is not evidence of real-world predictive reliability.

The online workflow is appropriate when one request must receive a response immediately, such as an interactive application or REST request.

Batch Inference

batch_inference.py constructs 20 new passenger records in a pandas DataFrame and calls the saved pipeline's predict() method once for the complete batch. It also calls predict_proba() once when supported. The output is written to batch_predictions.csv at the project root using a path derived from the script location.

Output structure

The verified CSV contains 20 data rows. It includes Batch_Record_ID, the seven input features, Predicted_Survived, Predicted_Label, Probability_Did_Not_Survive, and Probability_Survived. Missing Age and Embarked values are present in selected records and are handled by the persisted preprocessing pipeline.

Actual CSV sample

ID	Pclass	Sex	Age	Embarked	Prediction	Label	P(survive)
1	1	female	34.0	C	1	Survived	0.9440
2	3	male	22.0	S	0	Did not survive	0.0893
7	3	male	missing	S	0	Did not survive	0.0779
13	3	female	24.0	S	1	Survived	0.6142
20	3	female	12.0	missing	1	Survived	0.6201

Rows shown are selected directly from the generated CSV; the complete artifact remains batch_predictions.csv.

Batch inference is useful when records can be collected and processed together. It can improve throughput by reducing per-record orchestration overhead, but results are delayed until the batch is submitted.

Streaming Inference

streaming_inference.py simulates incoming data by iterating through exactly 10 newly constructed, distinct passenger records. For every event, it prints an arrival message, converts the dictionary into a one-row DataFrame with the seven expected feature names, calls predict() for that one record, and displays the class label and estimated probabilities.

Timing behavior

The script executes time.sleep(1) only when the current record is not the final record. Therefore, the simulation has nine one-second inter-record delays and no delay after record 10. The verified execution completed with exit code 0 and processed all 10 records.

Verified output examples

Record	Features summary	Prediction	Estimated probabilities
1	Pclass 1, female, age 62, fare 142, S	1 — Survived	0: 19.81%; 1: 80.19%
6	Pclass 2, male, age missing, fare 30, C	0 — Did not survive	0: 70.46%; 1: 29.54%
10	Pclass 3, male, age 26, fare 22.5, C	0 — Did not survive	0: 89.36%; 1: 10.64%

This simulates sequential incoming data rather than a finite group submitted together. It is useful for demonstrating event-by-event processing, although this assignment uses a local loop rather than an external message broker.

FastAPI Model Serving

The implemented app.py loads model.pkl during module initialization using a path derived from __file__. It does not train a fallback model. If loading fails, the application remains importable but reports that the model is unavailable through a safe HTTP 503 response and logs the underlying issue server-side.

Endpoints and validation

Endpoint	Behavior
GET /health	Returns HTTP 200 with {status: healthy, model_loaded: true} when the model is ready; otherwise HTTP 503.
POST /predict	Accepts a Pydantic PassengerRequest, creates a one-row DataFrame, calls the loaded pipeline, and returns prediction and probabilities.
GET /docs	FastAPI-generated Swagger UI for interactive API documentation and request testing.

Pclass, Sex, SibSp, Parch, and Fare are validated as required fields with constrained values or non-negative bounds. Age and Embarked are optional and nullable, matching the saved preprocessing pipeline's ability to impute missing values. Invalid requests are rejected by FastAPI/Pydantic with HTTP 422.

Reproduced API/client verification

Request	Observed result
GET /health	HTTP 200; model_loaded: true
Valid POST /predict	HTTP 200; class 1, Survived; probabilities returned
Invalid POST /predict with Pclass=4	HTTP 422 validation error

These results were reproduced with a temporary local Uvicorn server and client.py. The server was stopped after the check. No API screenshots were available in the project screenshots folder.

Results, Observations, and Comparison

Strategy	Records	Processing style	Typical use case	Demonstrated output
Online	1	One request at a time	Interactive or low-latency prediction	Class 1; survival estimate 84.82%
Batch	20	One vectorized prediction call	Scheduled or accumulated records	CSV with 20 rows and probabilities
Streaming	10	Sequential event-by-event	Continuous incoming events	10 arrivals; one-second inter-record delay

Assignment questions

1. How does Online Inference differ from Batch Inference?
Online inference handles one request and returns its result immediately. Batch inference waits for a group of records and processes them together, usually improving throughput but introducing batch waiting time.

2. What is the difference between Streaming Inference and Batch Inference?
Streaming inference processes each record as it arrives, while batch inference processes a finite collection together. The streaming demonstration intentionally introduces a one-second delay between incoming records.

3. Why is FastAPI used for model serving?
FastAPI provides a lightweight HTTP interface, automatic request validation through Pydantic, interactive OpenAPI/Swagger documentation, and a practical way for other applications to request predictions.

4. Why should the same trained model be reused for all inference strategies?
Reusing one fitted artifact keeps preprocessing, feature ordering, learned parameters, and prediction behavior consistent. Retraining separately for each strategy would make comparisons unreliable and could introduce data leakage or incompatible transformations.

Verified versus pending: Training, online, batch, streaming, and local API/client checks were executed. No screenshots were available. Production deployment, load testing, monitoring, and external-client integration remain outside this experiment.

Conclusion and References

This experiment established a reproducible path from an official Titanic dataset to a persisted machine-learning model and several serving patterns. The training script validates the data contract, imputes missing values, encodes categorical features, trains LogisticRegression inside a single Pipeline, evaluates the held-out split, and saves model.pkl. The online, batch, and streaming scripts demonstrate three different operational ways to reuse that exact artifact. The FastAPI service adds a validated REST boundary and the client confirms healthy, valid, and invalid request behavior.

Limitations

- The inference records are constructed demonstrations and do not establish real-world predictive reliability.
- The reported 80.45% accuracy is specific to the verified held-out split and should not be generalized without further evaluation.
- No production deployment, load test, model monitoring, or security hardening was performed.
- No screenshots were available in the project screenshots directory, so none were inserted.

References

1. Kaggle, “Titanic: Machine Learning from Disaster — Data.” https://www.kaggle.com/competitions/titanic/data
2. scikit-learn documentation, “Pipeline.” https://scikit-learn.org/stable/modules/compose.html
3. scikit-learn documentation, “ColumnTransformer.” https://scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html
4. FastAPI documentation. https://fastapi.tiangolo.com/
5. Pydantic documentation. https://docs.pydantic.dev/
6. Joblib documentation. https://joblib.readthedocs.io/

Submission status: The PDF report is generated from the inspected project files and verified outputs. The application scripts were not modified for report generation, and the model was not retrained.